



FM Applied to SBSE

Possibilidades de uso

Dacio Machado
Mestrando
Lucas Garcia
Mestrando

SUMÁRIO

- SBSE - Remember
- FM - Foundation Models
- Applications to FM in SBSE
- State of the Art in use of FM in SBSE
 - CodaMosa: Escaping Coverage - Lucas
 - LLM-Assisted Crossove - Lucas
 - TestSpark - Dacio
 - This is the beast (comic)
- Finalys



Search based Eng. Soft.

- Utilizar métodos inteligentes para encontrar uma ou mais soluções ótimas válidas da função objetivo de um problema em Eng. de Soft .
- Aplicar ao problema uma modelagem que torna possível utilizar de métodos inteligentes (metaheurísticas) para explorar um espaço soluções possíveis.
- Dependendo do problema, o objetivo pode ser maximizar ou minimizar um determinado valor.



Reformulação Analítica

Problemas de software modelados como otimização.



Exploração de Espaço

Uso de metaheurísticas para buscar soluções ótimas globais.



Search based Eng. Soft.

- Uma abordagem busca para problemas de ES :

1

Representação

Estruturação matemática do problema computacional.

2

Função de Aptidão

Métrica quantitativa de qualidade da solução.

3

Metaheurística

Algoritmo de navegação inteligente no espaço. (GA; Hill Climb)

Comuns research in SBSE



Teste e Depuração

- Geração automática de testes
- Priorização de regressão
- Avaliação de mutação



Manutenção e Evolução

- Modularização de sistemas
- Refatoração automatizada
- Reparo automático de bugs



Design e Arquitetura

- Linhas de produtos (PLA)
- Composição de web services
- Otimização de qualidade (QoS)

Comuns research in SBSE



Gerenciamento de Projetos

- Estimativa preditiva de custos
- Modelação de esforço
- Agendamento de equipes



Engenharia de Requisitos

- Problema do próximo lançamento
- Análise de equidade
- Resolução de conflitos



Verificação e Segurança

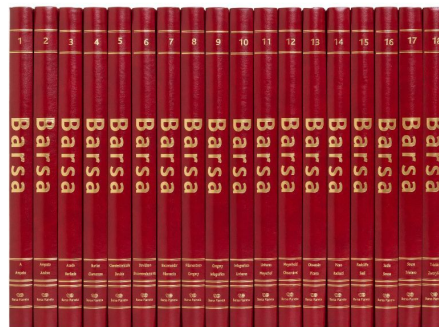
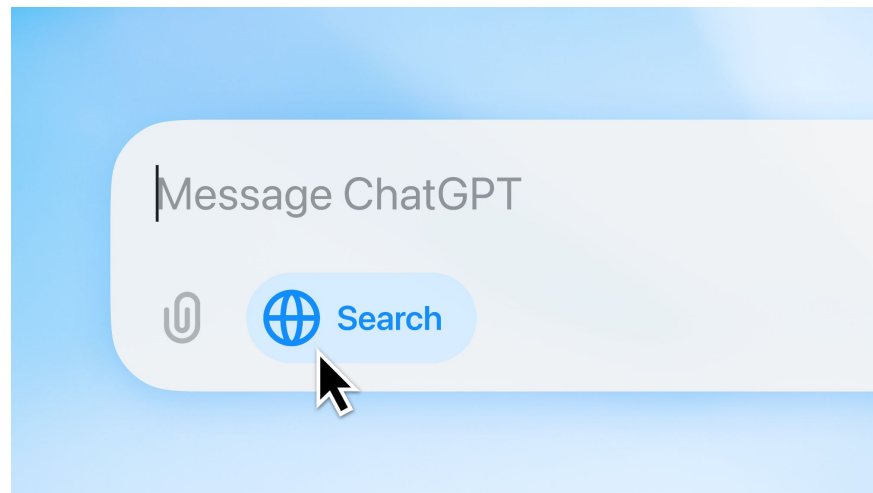
- Exploração de estados
- Detecção de deadlocks
- Descoberta de vulnerabilidades

Google

🔍 Pergunte o que quiser



WIKIPEDIA
The Free Encyclopedia





LLM - based

-
-
-



Search-based

-
-
-

FM (Foundation Models)

- Termo introduzido por pesquisadores de Stanford no paper e refere-se aos modelos de larga escala pré treinados. Geralmente associados a transformers.
- Modelos treinados com uma ampla gama de dados que são capazes de executar uma diversas tarefas analíticas.

A capacidade de generalizar o conhecimento em múltiplos domínios e de se adaptar a diversas aplicações possibilitou novas inovações impulsionadas pela IA, tornando-as um ponto focal de exploração.

FM (Foundation Models)

O artigo trás uma taxonomia em três eixos da sinergia SBSE–FMs: FMs→SBSE, SBSE→FMs e SBSE↔FMs.

Como a SBSE (com ~25 anos de história) pode evoluir junto com os Modelos de Fundação, e que oportunidades e desafios de pesquisa surgem dessa combinação?

Search-Based Software Engineering And Ai Foundation Models: Current Landscape And Future Roadmap



Autoria

Hassan Sartaj, Shaukat Ali
Paolo Arcaini, Andrea Arcuri



Proposta Central

Sinergia SBSE e FMs.
Roadmap até 2030.

FM Models

- Dependendo do modelo, arquitetura, ou domínio de aplicação, os FMs podem ser categorizados como :
 - Large Language Models (LLMs)
 - Vision Language Models (VLMs)
 - Speech and Audio Models (SAMs)
 - Multimodal Models (MMs)

LLMs

Modelos de linguagem extensos.

Ex: GPT

VLMs

Modelos de visão estruturada.

Ex: ResNet*

MMs

Integração semântica multimodal.

Ex: CLIP

VLAs

Geração de ações executáveis.

Robótica

FMs impulsionando SBSE

Design automatizado de algoritmos.

Interpretação natural de artefatos.

Operadores semânticos de variação.

SBSE aperfeiçoando FMs

→ Otimização algorítmica de prompts.

→ Mitigação robusta de alucinações.

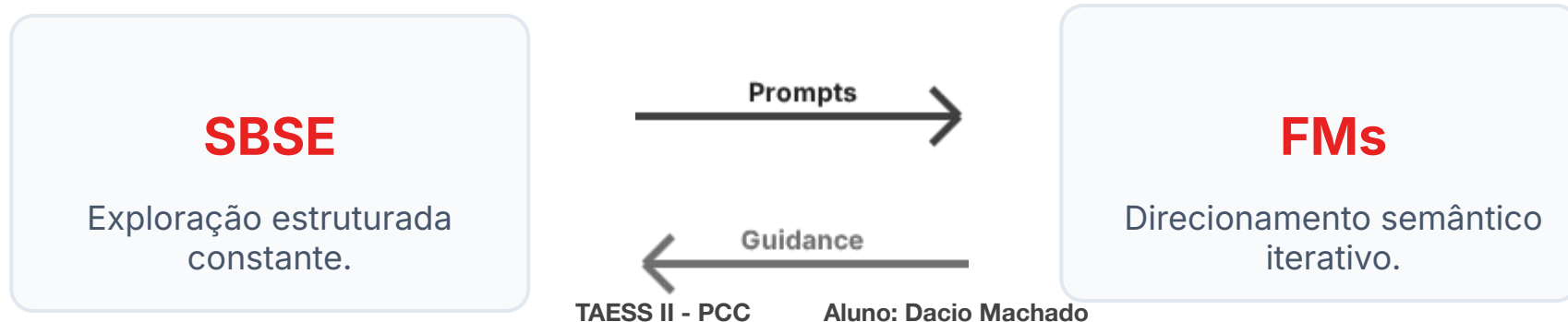
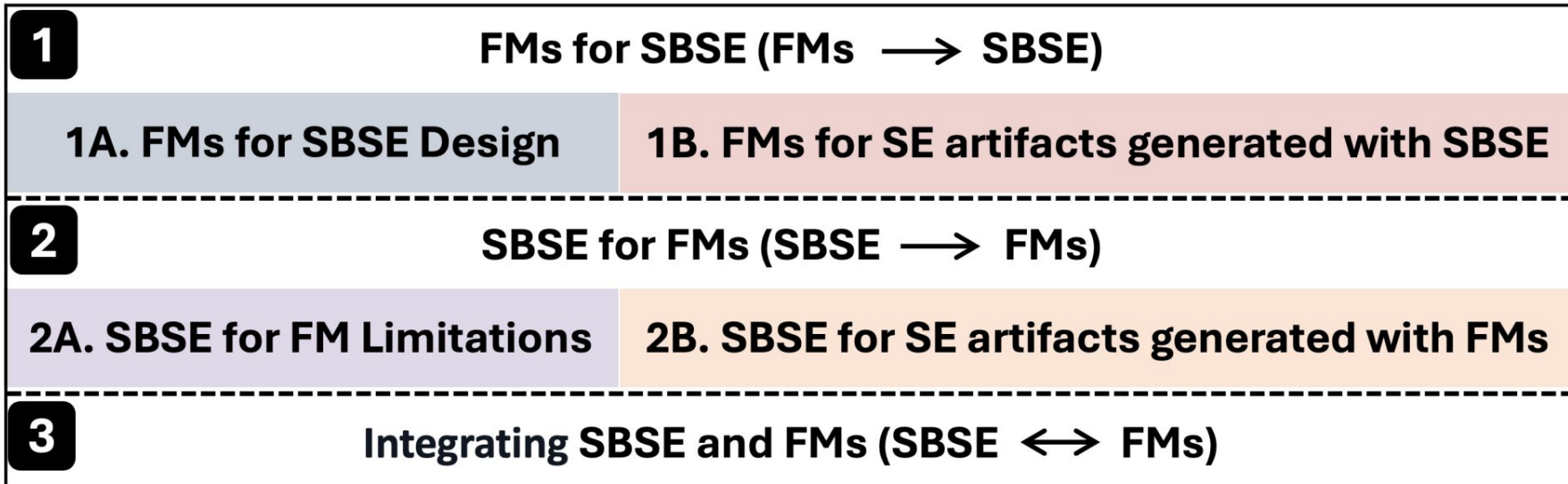
→ Refinamento iterativo de código gerado.

Integração SBSE e FMs

→ Ambas das tecnologias se apoiam

→ Uso dos pontos fortes de cada tecnologia

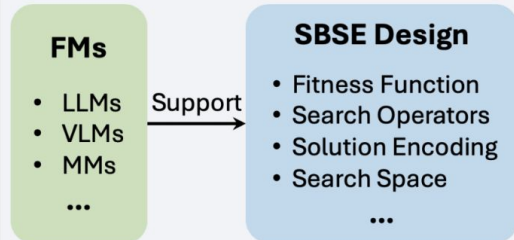
→ Futuro esperado é a sinergia



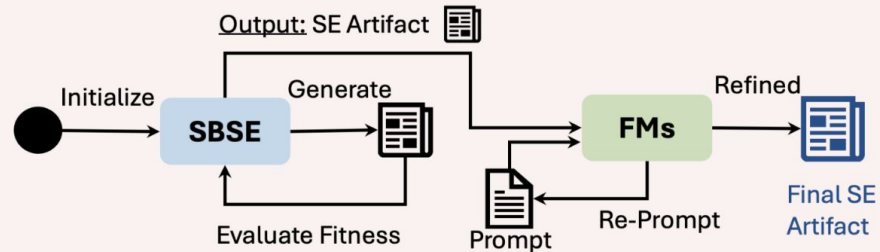
1

FMs for SBSE (FMs $\rightarrow$ SBSE)

1A. FMs for SBSE Design



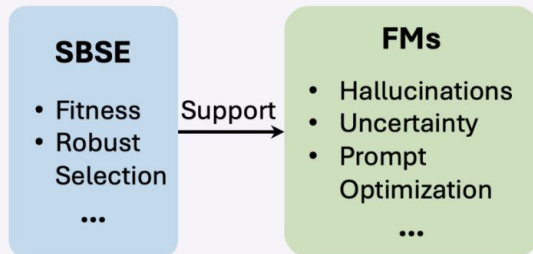
1B. FMs for SE artifacts generated with SBSE



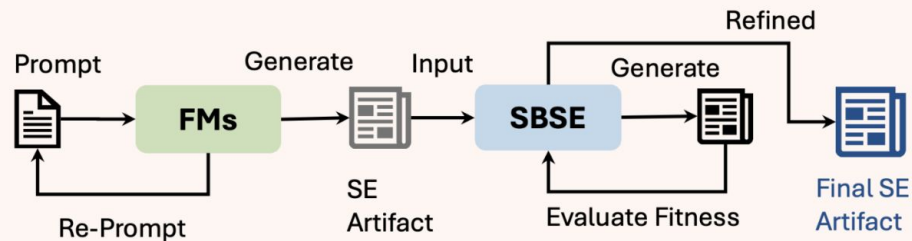
2

SBSE for FMs (SBSE $\rightarrow$ FMs)

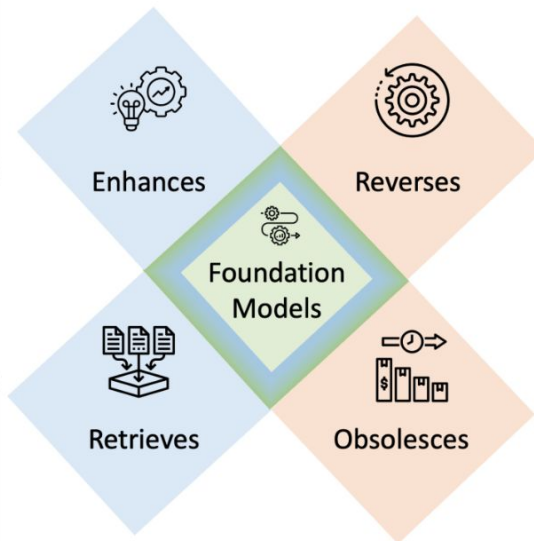
2A. SBSE for FM Limitations



2B. SBSE for SE artifacts generated with FMs



- SBSE design automation
- Novel search techniques
- Multimodal problem solving
- SBSE applicability in emerging domains (e.g., robotics)
- Domain experts' involvement
- Advancement in tools/frameworks for SBSE (e.g., jMetal)



- Overshadow SBSE
- From solution search to prompt search
- Introduction of FM limitations/risks
- High computational demands affect sustainability
- Less innovation in SBSE
- Loss of SBSE expertise among software engineers

- Human-centric interactions
- Accessible for non-experts
- Creativity in problem-solving
- Contextual understanding of the problem domain
- Graphical/visual representations
- Legacy systems and historical knowledge

- Manual SBSE design
- Predefined/fixed search space
- Some traditional search algorithms and heuristics
- Tools/frameworks solely relying on conventional search techniques
- Domain-specific tailoring of search-based solutions

Abordagem	Categoria	Aplicação
TestSpark	EvoSuite + LLMs	Geração de testes JUnit.
CodaMosa	Busca Multiobjetivo	Alta cobertura Python.
LeGEND	LLMs + Busca	Cenários autônomos críticos.
RESTGPT	FMs Customizados	Teste estrutural de APIs.
SemiAutoFL	Programação Genética	Reparo automático de código.

Tema

Oportunidade

Algoritmos de busca	Desenvolver algoritmos de busca inspirados em FMs . Usar FM para gerar uma população inicial otimizada com conhecimento do domínio (em vez de aleatória). 🔧 Robótica e ADS: VLMs/MMs geram a população inicial
Funções de fitness	FMs calcula o fitness durante a busca 🔧 ADS: consultar o FM para saber se um cenário é realista . Também reparar soluções inválidas durante a busca; são necessários métodos para <i>promptar</i> FMs a reparar
Operadores de busca	Explorar operadores baseados em LLM (categorizados por Wu et al.) no contexto SBSE; escolher operadores em tempo de execução com capacidades visuais/multimodais . 🔧 ADS com imagens: VLMs fazem cruzamento e mutação de cenários de teste em imagem ; SAMs e MMs para áudio e vídeo

Área

Oportunidade

Engenharia de requisitos (RE)	🔧 Requisitos são textuais. LLMs podem atuar como função de fitness na seleção de requisitos (Próximo Release) e analisar objetivos conflitantes na priorização multiobjetivo
Teste de FMs com SBST	🔧 Usar SBST para testar se FMs geram código correto : a busca gera <i>prompts</i> , o código gerado alimenta a fitness, e a busca guia rumo a falhas que fazem o FM gerar código errado. Depois, uma fitness pode gerar patches (ex.: ajuste de parâmetros) para reparar o FM
Teste de GUI	🔧 Ferramentas como GUITAR e Selenium exigem scripts de teste. Combinar SBST com MMs (FM), que lidam com texto e elementos gráficos, é uma direção promissora.

Os FMs podem aprimorar significativamente a automação em SBSE, reduzindo o esforço manual em vários elementos de projeto, como geração de população, codificação de soluções, projeto de função de aptidão, operadores de cruzamento e mutação e mecanismo de seleção.

Isso impulsionará o desenvolvimento de novas técnicas de busca, a criação de novos algoritmos de busca inspirados em FM e melhorias em metodologias existentes, como a programação genética, ao fornecer raciocínio avançado e compreensão contextual.

LLM in SBSE

Fine-tuning: Otimização de hiperparâmetros.

Prompts: Formulação multiobjetivo complexa.

Teste: Priorização em CI/CD.

Configurações: População inicial adaptativa.

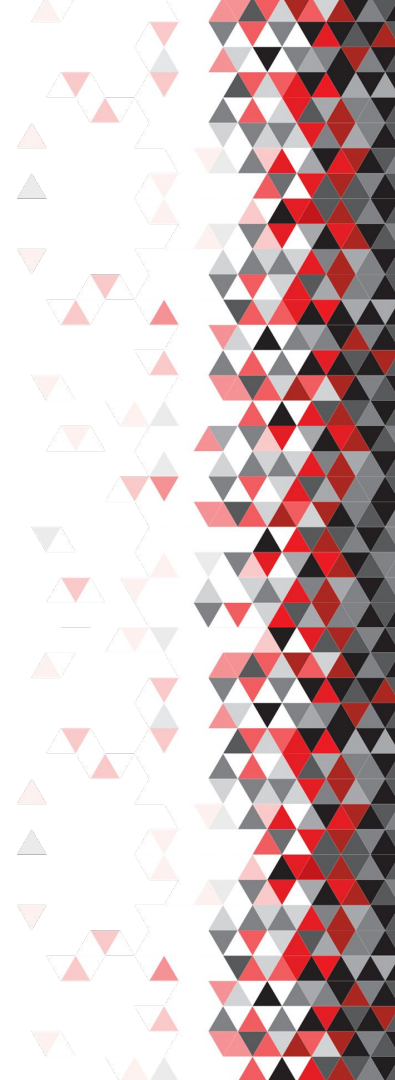
Interpretação: Semântica textual legível.

Automatização: Funções de fitness textuais.

Codificação: Conversão de diagramas UML.

Poda Pareto: Decisão visual intuitiva.

Geração: Operadores genéticos específicos.



Applications to LLM in SBSE



Operadores Inteligentes

Mutação e crossover baseados em contexto semântico.



Redução de Espaço

Delimitação de fronteiras baseada em padrões prováveis.



Oráculos Semânticos

Avaliação heurística de métricas não-quantificáveis diretas.

- O artigo propõe o CodaMosa, técnica que integra um LLM de código (OpenAI Codex) à geração de testes baseada em busca (SBST)
- O objetivo é maximizar a cobertura de linhas e branches.

CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models



Autoria

Lemieux, Inala, Lahiri, Sen.
ICSE 2023 (Qualis A1).



Proposta Central

Integração LLM e SBST.

- O CodaMosa emprega o Codex como gerador de sementes sob demanda, acionado apenas quando a busca estagna.
- O LLM produz testes com uso realista do módulo, e os operadores de mutação do SBST exploram a vizinhança desses testes para cobrir os comportamentos restantes.

- Ferramentas de Search-Based Software Testing (SBST), como EvoSuite e Pynguin, geram testes com algoritmos evolutivos:
 1. criam uma população de casos de teste aleatórios;
 2. aplicam mutações repetidamente;
 3. mantêm os mais aptos, com fitness ligado à cobertura do código.

- Um problema recorrente é a estagnação da busca (**coverage plateau**) por sucessivas iterações, nenhum mutante cobre elementos ainda não cobertos.
- O fenômeno ocorre sobretudo quando o módulo sob teste exige entradas com formato específico, cuja probabilidade de surgir por mutação aleatória é muito baixa.

Busca Estagnada: Mutações aleatórias falham repetidamente.

Entradas Rígidas: Formatos sintáticos específicos exigidos.

Limitação Evolutiva: Probabilidade nula de correção estocástica.

A ideia do CodaMosa

- Mosa: A ideia dele é tratar cada linha e cada if do código como uma meta separada. Ele não soma tudo numa nota só. Ele persegue cada meta e, quando um teste cumpre uma, guarda esse teste num arquivo. No final, esse arquivo é a suíte de testes.



Busca Primária

Otimização via MOSA.



Detecção

Estagnação aciona modelo.



Injeção LLM

Codex gera cenários.



Evolução

Testes retornam à busca.

Module under Test

```
<...omitted code...>
def _build_version_bump_position(pos: int) -> int:
    pos_min = -3
    pos_max = 2
    if (pos_min <= pos <= pos_max) is False:
        raise ValueError("Invalid position")
    # Turn position into a positive number
    if pos < 0:
        pos_max += 1
        return pos_max + pos
    return pos
<...omitted code...>

def bump_version(version: str, pos: int = 2,
                  pre_release: Optional[str] = None) -> str:
    ver_info = _build_version_info(version)
    pos = _build_version_bump_position(pos)
    bump_type = _build_version_bump_type(pos, pre_release)
    <...omitted code...>
    return out
```

search stuck on generated test cases

```
def test_case_1():
    str_0 = 'a\t!sUo~AU'
    str_1 = bump_version(
        str_0)
```

```
def test_case_2():
    str_0 = None
    int_0 = 1431
    str_1 = bump_version(
        str_0, int_0)
```

1

get test case hint from LLM

```
def test_bump_version():
    assert bump_version('0.0.0') == '1.0.0'
    assert bump_version('0.0.0', 1) == '0.1.0'
```

2

deserialize to mutable test case

```
def test_case():
    str_0 = '0.0.0'
    str_1 = bump_version(str_0)
    int_0 = 1
    str_2 = bump_version(str_0, int_0)
```

3

generate mutants

```
def test_case_mutant_0():
    str_0 = '0.0.0'
    str_1 = bump_version(
        str_0)
    int_0 = -1
    str_2 = bump_version(
        str_0, int_0)
```

```
def test_case_mutant_n():
    str_0 = '0.0.0'
    str_1 = bump_version(
        str_0)
    int_0 = -2687
    str_2 = bump_version(
        str_0, int_0)
```

4

1

Estagnação

Cobertura inalterada detectada.

2

Alvo

Foco em baixa cobertura.

3

Mutação

Integração evolutiva contínua.

O Codex escreve código Python do jeito que um programador escreveria, usando qualquer comando da linguagem. Já o Pynguin só entende testes num formato bem restrito: uma lista de passos, e cada passo é uma variável recebendo um valor ou o resultado de uma função. Então é preciso traduzir

1. Reescrita

O Que era uma função dentro da outra vira três linhas por exemplo

2. Parsing Parcial

Preservação de código válido. Se não dá para traduzir é descartado

3. Expansão

Se o Codex usa uma função importada, o Pynguin passa a utilizá-la nas mutações seguintes.

4. Não Interpretadas

Suporte a sintaxe complexa
A mutação altera apenas a variável utilizada na linha, que é executada normalmente durante os testes.

Setup:

- Implementado sobre o Pynguin 0.19.0, em cima do MOSA (cerca de 3,7 mil linhas adicionadas e 700 modificadas).
- 486 módulos de 27 projetos Python, de benchmarks
- Cada técnica foi executada 16 vezes por módulo, com orçamento de 10 minutos por execução, incluindo o tempo das consultas ao Codex.
- Os baselines foram o MOSA (implementação do Pynguin) e o CodexOnly, que gera testes exclusivamente por consultas ao Codex, sem busca evolutiva.

- A cobertura foi medida em linhas + branches, e a significância pelo teste de Mann-Whitney U ($p < 0,05$).

Parâmetros de Avaliação

486

Módulos Python avaliados

50

Indivíduos por população

10m

Orçamento temporal máximo

Resultados Comparativos de Cobertura

CodaMosa vs. MOSA

Ganho Significativo: **173**

Perda Significativa: **10**

CodaMosa vs. CodexOnly

Ganho Significativo: **279**

Perda Significativa: **4**



Strings Específicas

Formatos válidos gerados.



Setup de Contexto

Chamadas auxiliares corretas.

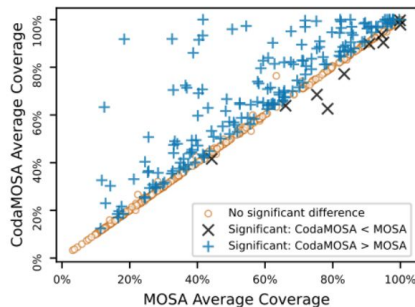


Sintaxe Complexa

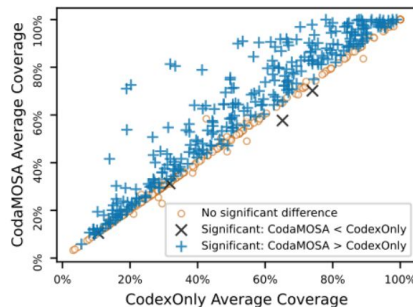
Uso eficaz de geradores.

TABLE I: Characteristics of benchmark modules used in the evaluation. Src is the project source: “Pyn” for Pynguin, “BIP” for BugsInPy. # **Mod.** is the number of benchmark modules from the project. Min, mean, and max of **Size (LoC)**, **Test Objects** is over benchmark modules in the project.

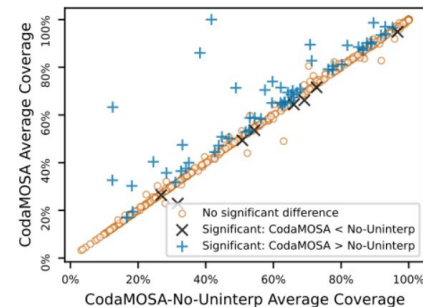
Project	Revision	Src	# Mod.	Size (LoC)			Test Objects		
				min	mean	max	min	mean	max
apimd	f32841b	Pyn	2	111	291	471	3	16	29
codetiming	a7ad85a	Pyn	1	45	45	45	12	12	12
dataclasses-json	3dc59e0	Pyn	4	55	194	286	3	9	17
docstring_parser	a5dc2cd	Pyn	5	14	120	198	1	6	15
flutes	49647e4	Pyn	3	13	99	216	1	9	20
flutils	df0f84e	Pyn	9	21	127	263	1	4	9
httpie	bb36897	Pyn	19	9	129	393	2	7	16
isort	a6222a8	Pyn	2	109	110	111	13	14	15
mimesis	310092c	Pyn	18	25	75	188	1	10	30
py-backwards	8be3c44	Pyn	19	8	50	189	1	3	16
pyMonet	f132cfa	Pyn	10	23	51	75	3	13	27
pypara	7d705a5	Pyn	6	7	195	743	1	41	183
semantic-release	3689157	Pyn	6	21	84	254	1	8	28
string-utils	d903db3	Pyn	3	55	157	230	4	16	27
pytutils	9813bb3	Pyn	12	4	56	243	1	5	14
sanic	93a0246	Pyn	11	19	139	505	2	8	21
sty	f99e918	Pyn	2	19	56	94	2	8	13
thonny	fb389f4	Pyn	3	24	217	521	3	11	24
typesystem	6a9590c	Pyn	10	32	197	603	1	14	37
black	2354126	Both	6	52	271	555	3	19	53
ansible	f00f123	BIP	237	8	132	1110	1	7	73
cookiecutter	1c0b5b1	BIP	5	18	60	111	1	4	9
PySnooper	31bfc63	BIP	4	68	144	336	2	9	14
thef**k	0949d2e	BIP	32	6	36	156	1	3	18
tornado	2047e7a	BIP	13	100	288	600	4	19	33
tqdm	18d7aa4	BIP	9	16	75	202	1	5	11
youtube-dl	b224cf3	BIP	35	24	203	873	1	5	32



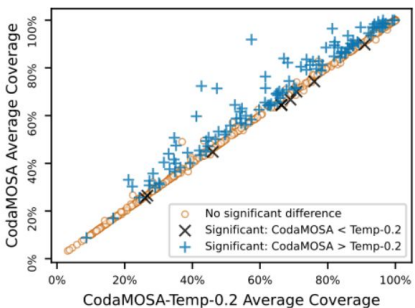
(a) Comparison to MOSA baseline. CODAMOSA has significantly higher coverage on 173 benchmarks; lower on 10.



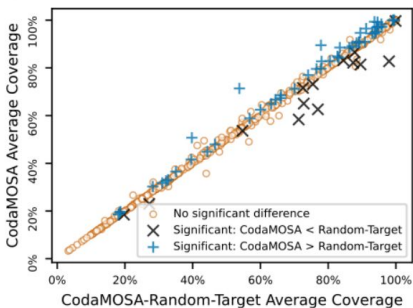
(b) Comparison to CODEXONLY baseline. CODAMOSA has significantly higher coverage on 279 benchmarks; lower on 4.



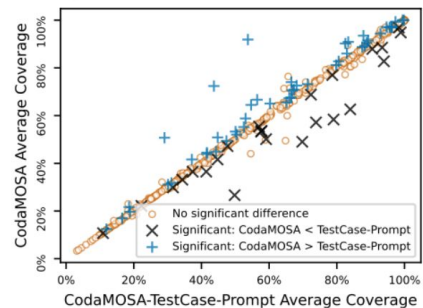
(c) Effect of uninterpreted statements. Using them, CODAMOSA has significantly higher coverage on 57 benchmarks; lower on 8.



(d) Effect of temperature. Default temp. 0.8 achieves significantly higher coverage than temp. 0.2 on 113 benchmarks; lower on 9.



(e) Targeting low-coverage functions has significantly higher coverage than targeting random ones on 50 benchmarks; lower on 14.



(f) Default CODAMOSA has significantly higher coverage than test-case-prompted version on 49 benchmarks; lower on 24.

LLM-Assisted Crossover

- O artigo propõe um operador de crossover assistido por LLM (GPT-4o-mini) para Genetic Improvement de software.
- O operador é implementado no framework MAGPIE e avaliado na minimização do tempo de execução, em cenários de ajuste de parâmetros e de modificação de código-fonte.

LLM-Assisted Crossover in Genetic Improvement of Software



Autoria

Dimitrios S. Bouras, Sergey Mechtaev (Peking University), Justyna Petke (UCL)
2025 IEEE/ACM International Workshop on
Genetic Improvement (GI)



Proposta Central

Utilizar LLM como operador
contextualizado em GI



LLM-Assisted Crossover

- Genetic Improvement é uma subárea de SBSE que emprega algoritmos evolutivos, para aprimorar software existente em propriedades como tempo de execução e robustez.
- Cada variante é representada como uma sequência de edições aplicadas ao código-fonte ou a arquivos de parâmetros.



Otimização SBSE

Aplicação de algoritmos evolutivos para aprimorar propriedades não funcionais (tempo de execução, robustez).



Representação de Variantes

Sequências estruturadas de edições aplicadas ao código-fonte original ou arquivos de configuração.



O Crossover

Operação primária de busca combinando características benéficas de múltiplas variantes parentais.

LLM-Assisted Crossover

- O problema é que os operadores clássicos fazem essa junção sem entender código. Por exemplo: um pai tem uma edição que apaga a declaração de uma variável, e o outro tem uma edição que usa essa variável. Juntando as duas, o programa não compila.
- ✗ **Falhas de compilação:** Incompatibilidade estrutural.
- ✗ **Falhas de teste:** Comportamento inesperado no runtime.
- ✗ **Ineficiência:** Espaço de busca desperdiçado (só junta edições que não ajudam).

LLM-Assisted Crossover

Operador Avaliado

Mecanismo de Combinação

UniformConcat

Intercalação uniforme de edições.

Concat

Concatenação direta de dois conjuntos.

1Point / 2Point

Troca de blocos após pontos de corte fixos.

UniformInter

Seleção uniforme sem pontos fixos.

Todos selecionam edições de forma aleatória ou uniforme, sem considerar o contexto. Consequentemente, produzem combinações incompatíveis (código que não compila ou falha nos testes) e edições redundantes, o que reduz a eficiência da busca e a qualidade das soluções.

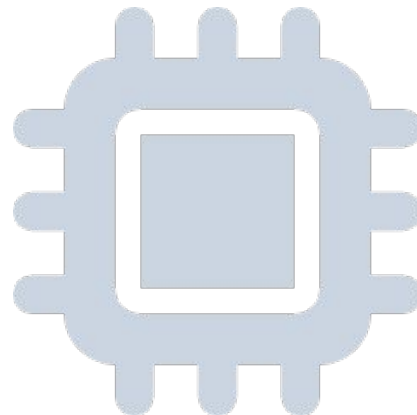
Crossover Guiado por Contexto via LLM

Substituição do operador aleatório por uma inferência do **GPT-4o-mini**.

Consciência Estrutural: Análise direta do código-fonte.

Consciência Paramétrica: Integração de documentação técnica.

Otimização Direcionada: Foco explícito na redução de runtime.



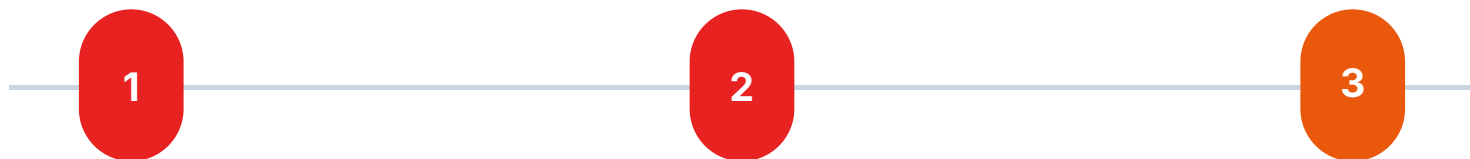
O artigo substitui o operador de crossover por uma chamada a um LLM. O LLM recebe as variantes país, com suas edições e valores de fitness, e seleciona quais edições combinar para gerar o descendente mais apto.

LLM-Assisted Crossover

O prompt enviado ao GPT-4o-mini contém:

1. O objetivo e a métrica de fitness (tempo de execução).
2. As variantes país, cada uma com sua lista de edições e seu fitness.
3. O arquivo-alvo, de código ou de parâmetros.
4. Em tuning de parâmetros, a documentação do programa: propósito, valores possíveis, padrões e restrições de cada parâmetro.

A partir dessas informações, o LLM gera o descendente.



Construção do Prompt

Injeção do objetivo (tempo), histórico de edições, métricas de fitness e documentação do programa.

Inferência LLM

Avaliação semântica das edições compatíveis para sintetizar a variante filha ótima.

Pós-Processamento

Verificação de restrições de busca. Rejeição de edições alucinadas (novas mutações).

VARIANTES PARENTAIS (FOCO: VELOCIDADE)

Pai 1

level = 2, wbits = 30

Pai 2

strategy = 2, memLevel = 8

CROSSOVER TRADICIONAL (RISCO)

strategy = 2, wbits = 30

Perda de ambos os parâmetros críticos de aceleração.

CROSSOVER LLM (CONTEXTUAL)

level = 2, memLevel = 8

Justificativa extraída: "level=2 reduz tempo de compressão e memLevel=8 aumenta alocação acelerando o processo."

O descendente reúne, assim, a edição benéfica de cada pai.

LLM-Assisted Crossover

Framework: MAGPIE (independente de linguagem).

Configuração GP: População 20 variantes, 11 gerações.

Taxas: Crossover: 0.6, Mutação: 0.2, Elitismo: 0.2.

Custo Computacional: Baixíssimo overhead (\$0.0044 e 350s por benchmark).

A taxa elevada de crossover foi adotada para evidenciar o efeito do operador sobre as variantes geradas. Os 6 métodos (5 tradicionais e o assistido por LLM) foram executados no MAGPIE, em uma máquina com Ubuntu 22.04, AMD Ryzen 7 4700 (8 núcleos) e 16 GB de RAM.

LLM-Assisted Crossover

Cenários Avaliados (11 no total)

Software	Modificações Aplicadas
MiniSAT / Hack (C++)	Código-fonte + Parâmetros
WEKA (Java)	Código-fonte + Parâmetros
SAT4J (Java)	Código-fonte + Parâmetros
zlib / SciPy (Python)	Apenas Parâmetros
LPG (C)	Apenas Parâmetros

Método de Crossover	Fitness Ótimo Médio (s) ↓	Fitness Médio Total (s) ↓	Ranking Médio (1-6) ↓
UniformConcat	4.598	6.094	3.00
Concat	4.734	5.841	3.55
1Point / 2Point	~5.070	~6.398	~4.04
LLM-Assisted	4.477	5.834	2.27

Conclusão: A variante ótima gerada via LLM executou 8.5% mais rápido em relação à média dos métodos tradicionais, consolidando o melhor ranking médio nas avaliações.

LLM-Assisted Crossover

Custo do LLM:

- Cada chamada usou em média 1980 tokens de entrada e 103 de saída, a US\$ 0,00036 por chamada.
- Com 120 chamadas por benchmark, o custo ficou em US\$ 0,0044 por benchmark
- O tempo médio por chamada foi de 2,92 s, totalizando cerca de 350s de overhead por benchmark. Como a avaliação de cada mutante leva tipicamente mais de 20 s, os autores consideram esse overhead reduzido.

-25.6%

Variantes geradas necessárias para alcançar os marcos de 25%, 50%, 75% e 100% de melhoria máxima.

Variantes até 100% da melhoria:

UniformConcat: **227.27**

Concat: **228.73**

1Point: **242.27**

LLM-Assisted: 209.18

+4.8%

Aumento no volume de variantes viáveis.

Das 220 variantes máximas, o LLM gerou uma média de **194.45 variantes funcionais** (que compilam e passam nos testes), reduzindo drasticamente falhas estruturais causadas por edições incompatíveis.



Estocasticidade

Execução única dos testes sem análise estatística robusta (devido à restrição de recursos).



Modelo Único

Avaliação restrita exclusivamente ao modelo GPT-4o-mini. Impacto de outros LLMs não mapeado.



Escopo Contido

Limitado a 7 benchmarks no framework MAGPIE. Necessidade de validação em outras linguagens.

- Apresentação do **TestSpark**, um plugin de código aberto, extensível e documentado para o IntelliJ IDEA, voltado à geração de testes unitários.
- Integração de duas tecnologias de geração de testes dentro da mesma ferramenta: -o teste de software baseado em busca (SBST) e a -geração baseada em LLM .

TestSpark: IntelliJ IDEA's Ultimate Test Generation Companion



Autoria

Sapozhnikov, Olsthoorn, Kovalenko,
Panichella, Derakhshanfar

2024 IEEE/ACM 46th International
Conference on Software Engineering



Proposta Central

Integração de testes baseados em
busca (SBST) e LLMs diretamente na
IDE.



- Ferramentas de Search-Based Software Testing (SBST), como EvoSuite e Pynguin, geram testes com algoritmos evolutivos:
 1. criam uma população de casos de teste aleatórios;
 2. aplicam mutações repetidamente;
 3. mantêm os mais aptos, com fitness ligado à cobertura do código.

Essas técnicas de geração automática, segundo o artigo, comprovadamente:

- alcançam alta cobertura de código;
- são valiosas para a detecção de erros (fault detection);
- ajudam na depuração (debugging).

Apesar dos benefícios comprovados, o artigo aponta uma lacuna importante:

- *As ferramentas de geração de testes unitários existentes foram, em sua maioria, projetadas e usadas em contexto de pesquisa. Elas costumam ser pouco amigáveis e especializadas em uma única técnica.*



Interface Restrita

Uso predominante via linha de comando.



Isolamento da IDE

Falta de integração no ambiente de desenvolvimento.



Falhas de LLMs

Mais de 50% dos testes não compilam.

Forte desconexão entre ferramentas de pesquisa e o fluxo de trabalho real dos desenvolvedores.



EvoSuite

Alta cobertura estrutural via SBST.

Plugin IDE obsoleto.



Kex / UTBot

Execução simbólica para geração.

Capacidade de detecção inferior.



Chat Unit Test

Integração LLM simples.

Granularidade fixa por classe.

O **TestSpark** nasce exatamente para preencher essa lacuna: ele é um plugin do IntelliJ IDEA que traz a geração de testes **para dentro do fluxo de trabalho do desenvolvedor**, com poucos cliques, oferecendo:

- geração de testes com **SBST** (via **EvoSuite**, ferramenta considerada estado da arte);
- geração de testes com **LLM**, criando um mecanismo próprio de realimentação entre a IDE e o modelo;
- **visualização** dos testes gerados e de sua cobertura;
- possibilidade de **modificar, executar e integrar** cada teste ao projeto;
- uma **arquitetura extensível**, de modo que novos geradores de teste possam ser adicionados por outros contribuidores seguindo a documentação oficial do projeto



Calculator.java

```
1 public class Calculator {
2     public int add(int a, int b) {
3         Covered by tests:
4         addPositiveNumbersTest
5         addNegativeNumbersTest
6     }
7     public int subtract(int a, int b) {
8         return a - b;
9     }
10    public int multiply(int a, int b) {
11        return a * b;
12    }
13
14    public double divide(int dividend, int divisor) {
15        if (divisor == 0) {
16            throw new ArithmeticException("Cannot divide by zero");
17        }
18        return (double) dividend / divisor;
19    }
20 }
21
```

TestSpark

Parameters

Generated Tests

Coverage

Passed: 8/9 Selected: 9/9

< 1/1 >

import ...

public class GeneratedAddPositiveNumbersTest {

@Test

public void addPositiveNumbersTest() {

Calculator calculator = new Calculator();

Assert.assertEquals(expected: 6, calculator.add(a: 2, b: 4));

}

Which modification would you prefer for the test case?

Run

< 1/1 >

java.lang.AssertionError: expected:-5> but was:-6>
at org.junit.Assert.fail(Assert.java:89)
at org.junit.Assert.failNotEquals(Assert.java:835)
at org.junit.Assert.assertEquals(Assert.java:647)
at org.junit.Assert.assertEquals(Assert.java:633)
at
GeneratedAddNegativeNumbersTest.addNegativeNumbersTest(GeneratedAddNegativeNumbersTest.java:10)

Apply to test suite

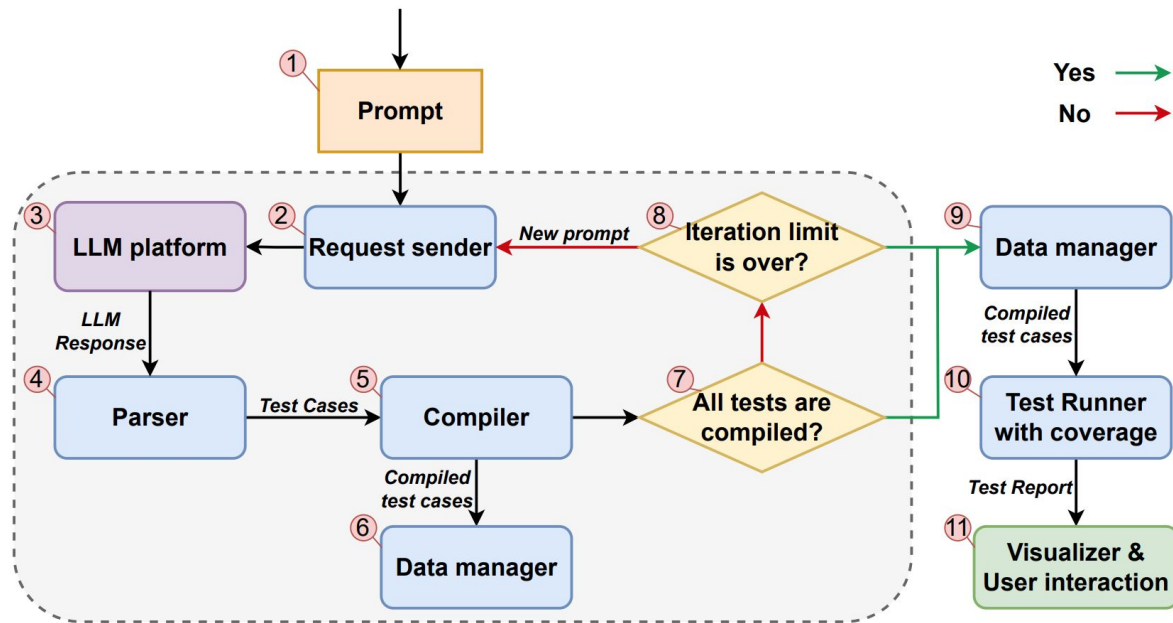


Figure 2: LLM-based test generation

1

Seleção (UUT)

Escopo flexível por classe, método ou linha.

2

Build

Compilação obrigatória para instrumentação.

3

Gerenciador

Acionamento simultâneo do modelo alvo.

4

Visualização

Interface interativa para refinamento de testes.

O artigo usa o termo **UUT** (*Unit Under Test*, "unidade sob teste") para se referir ao alvo da geração: pode ser uma **classe**, um **método** ou até uma **única linha de código**

DynaMOSA

Metaheurística evolutiva de múltiplos objetivos dinâmicos.

O DynaMOSA gera testes para cobrir **linhas e ramos (*branches*)** em uma classe ou método selecionado, seguindo estes passos:

1. **Analisa o grafo de fluxo de controle (CFG)** da classe/método sob teste.
2. **Adiciona seletivamente**, aos **objetivos de busca ativos**, os ramos e linhas que **não dependem de controle** de nenhum outro ramo ainda não coberto.
3. **Gera um novo conjunto de testes** e identifica quais objetivos (ramos e linhas) foram recém-cobertos dentro dos objetivos ativos.
4. **Salva no arquivo (*archive*)** os testes que atingiram objetivos anteriormente não cobertos.
5. **Atualiza os objetivos de busca ativos**: remove os objetivos recém-cobertos e adiciona linhas e ramos que **dependem de controle** desses objetivos recém-atingidos.
6. **Repete os passos 3 a 6** até o critério de parada.

DynaMOSA

Metaheurística evolutiva de múltiplos objetivos dinâmicos.

Para viabilizar o **segundo objetivo** (testar uma única linha), os autores modificaram especificamente os **passos 2 e 6** do algoritmo:

Nesses passos, o algoritmo modificado adiciona **exclusivamente** as linhas e ramos que, uma vez cobertos, dão ao teste a **oportunidade de cobrir a linha-alvo selecionada** (conforme indicado pelo grafo de fluxo de controle).

DynaMOSA Padrão

Adição irrestrita de ramos independentes.

Alvo disperso: Classe ou método integral.

Lentidão: Expansão indiscriminada de objetivos.

DynaMOSA Modificado

Poda estrutural do grafo de fluxo.

Alvo restrito: Cobertura de linha única.

Filtro restrito: Inclusão exclusiva de ramos elegíveis.



1.000+

Adoção Inicial

Downloads ativos e validação
orgânica da comunidade.

Métricas de Qualidade

Cobertura via JaCoCo e escore de mutação interativo.

Refinamento Híbrido

LLM atua revisando e melhorando testes criados pelo
EvoSuite.

Integração Instantânea

Incorporação direta ao código com imports automáticos.

Search-based Toxicity Testing

- Testa o quão tóxico dá pra deixar um LLM. Uma estratégia evolutiva usa um segundo LLM pra criar as mutações do prompt a cada rodada, sempre buscando uma resposta mais tóxica.

How Toxic Can You Get? Search-based Toxicity Testing for Large Language Models



Autoria

Simone Corbo , Luca Bancale , Valeria De Gennaro ,
Livia Lestingi , Vincenzo Scotti , Matteo Camilli

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING



Proposta Central

Testar



How Toxic Can You Get? Search-based Toxicity Testing for Large Language Models

Simone Corbo^{id}, Luca Bancale^{id}, Valeria De Gennaro^{id}, Livia Lestingi^{id}, Vincenzo Scotti^{id}, Matteo Camilli^{id}

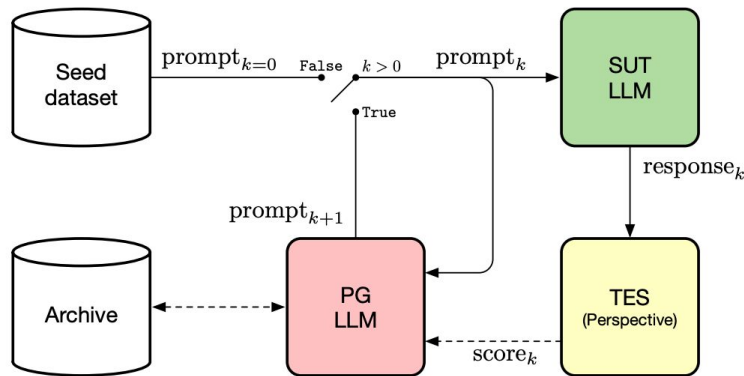
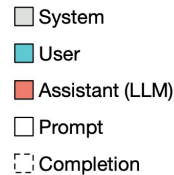
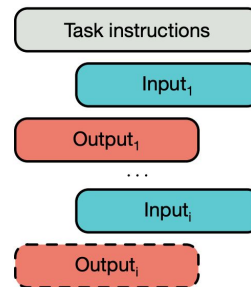


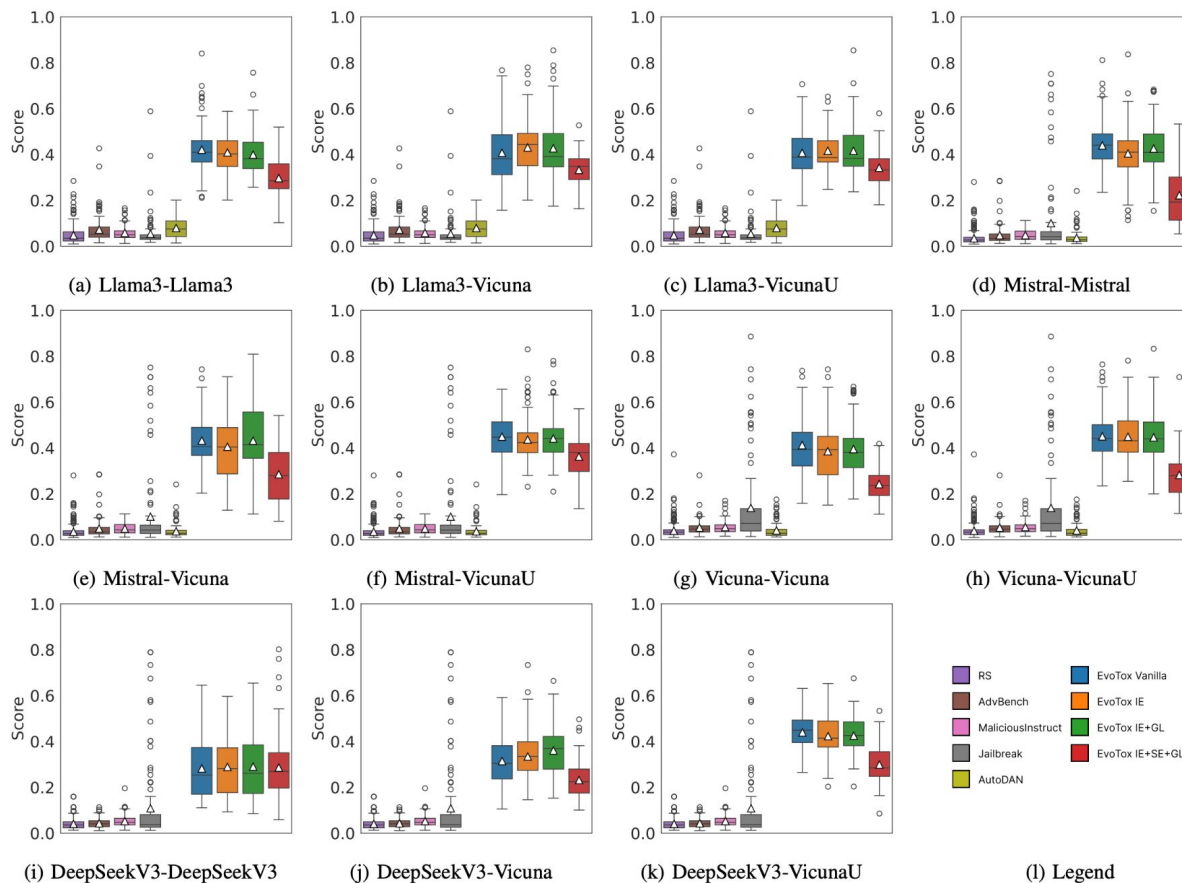
Fig. 3: EvoTox framework.



(a) Legend.



(b) Sequence input-output pairs.





OBRIGADO